
ASP.NET Core con .NET 10

Manual completo para crear APIs HTTP con Minimal APIs y Controllers, diseñar REST, conectar Entity Framework Core y elegir alternativas no REST

Finalidad

Pensado para alumnado que conoce los fundamentos de C# y quiere aprender a construir servicios web mantenibles: desde HTTP, routing y middleware hasta validación, persistencia, seguridad, tiempo real y comunicación entre sistemas.

Al terminar este manual podrás:

- Explicar qué aporta **ASP.NET Core** y recorrer la canalización completa de una petición HTTP.
- Crear APIs con **Minimal APIs** y con controladores, y elegir el enfoque adecuado.
- Diseñar recursos, rutas, operaciones, representaciones y contratos REST coherentes.
- Validar entradas, devolver códigos y cabeceras correctos y normalizar errores con **ProblemDetails**.
- Aplicar inyección de dependencias, servicios, configuración tipada, logging y cancelación.
- Persistir datos mediante **Entity Framework Core 10**, SQLite, migraciones y consultas eficientes.
- Conocer cuándo convienen RPC, webhooks, SignalR, WebSockets o gRPC en lugar de REST.
- Documentar, proteger, probar, observar y publicar una API preparada para evolucionar.

Edición didáctica 2026 · .NET 10

Índice

1. Mapa del manual y proyecto conductor	7
Capas del aprendizaje	7
Estructura propuesta	7
2. Qué es ASP.NET Core y qué problema resuelve	8
El problema antes del endpoint	8
Qué no es	8
3. HTTP como lenguaje de la API	9
Anatomía de la petición	9
Propiedades que condicionan el diseño	9
4. Crear el proyecto y dominar las herramientas	10
Plantillas	10
Comprobar sin depender del navegador	10
Actividades propuestas	10
5. Host, Kestrel, middleware, routing y endpoints	11
Middleware o filtro	11
6. Primera API con Minimal APIs	12
Por qué TypedResults	12
7. Routing y model binding en Minimal APIs	13
Fuentes de datos	13
Restricciones de ruta	13
8. Organizar Minimal APIs que crecen	14
Filtros de endpoint	14
9. Crear una API con Controllers	15

10. Minimal APIs frente a Controllers	16
11. Diseñar recursos y rutas REST	17
Relaciones y acciones	17
Reglas prácticas	17
12. Métodos HTTP y semántica correcta	18
PUT no es un PATCH grande	18
Creación correcta	18
13. Códigos de estado y cabeceras	19
Cabeceras útiles	19
14. DTO, contratos y mapeo	20
Proyección en la consulta	20
15. Validación en .NET 10	21
Minimal APIs	21
Controllers	21
16. Errores coherentes con ProblemDetails	22
Política de excepciones	22
17. Inyección de dependencias y servicios	23
18. Configuración, Options y secretos	24
Prioridad y secretos	24
19. Logging, trazas y cancelación	25
Logging estructurado	25
CancellationToken	25
Correlación	25
20. Conectar Entity Framework Core 10	26
21. CRUD con Minimal APIs y EF Core	27

Consulta y detalle	27
Actualizar y borrar	27
22. CRUD con Controllers y un servicio	28
Resultado de aplicación	28
23. Relaciones, transacciones y migraciones	29
Relaciones	29
Transacciones	29
Migraciones en equipos	29
24. Consultas eficientes, paginación y búsqueda	30
Reglas de rendimiento	30
25. Concurrencia e idempotencia	31
Claves de idempotencia	31
26. OpenAPI y documentación del contrato	32
Qué documentar	32
27. Versionar y evolucionar una API	33
Retirada	33
28. Seguridad básica de una API	34
Autenticación y autorización	34
CORS	34
Checklist	34
29. Rate limiting, caché y resiliencia	35
Caché HTTP	35
Llamadas salientes	35
30. Pruebas de servicios y endpoints	36
Pirámide práctica	36

Base de datos de prueba	36
Actividades propuestas	36
31. REST no es la única opción	37
32. RPC: operaciones como contrato	38
Diseñarlo bien	38
33. Webhooks: notificar eventos a otros sistemas	39
Receptor seguro	39
Reglas de entrega	39
34. SignalR: tiempo real de alto nivel	40
Diseño operativo	40
35. WebSockets: canal bidireccional de bajo nivel	41
36. gRPC: contratos tipados y streaming	42
Tipos de llamada	42
37. Elegir REST, RPC, webhooks, SignalR, WebSockets o gRPC	43
38. Publicación, salud y operación	44
Liveness y readiness	44
Publicar	44
39. Proyecto integrador: Biblioteca API	45
Requisitos	45
Criterios de aceptación	45
40. Diagnóstico de errores frecuentes	46
Método de diagnóstico	46
41. Checklist de calidad	47
Contrato HTTP	47
Código y datos	47

Operación	47
---------------------	----

42. Glosario	48
---------------------	-----------

43. Referencias	49
------------------------	-----------

1. Mapa del manual y proyecto conductor

El manual construye una API de biblioteca. Libros y préstamos sirven para practicar contratos HTTP, validación, persistencia y concurrencia sin ocultar las decisiones importantes detrás de una aplicación demasiado grande.

Capas del aprendizaje

Bloque	Pregunta que resuelve	Resultado
Fundamentos	¿Cómo llega una petición al código?	host, Kestrel, middleware, routing y endpoint
HTTP y REST	¿Qué contrato verá el cliente?	rutas, verbos, estados, cabeceras y representaciones
Implementación	¿Minimal API o controlador?	endpoints organizados, binding, DTO y respuestas tipadas
Aplicación	¿Dónde viven las reglas?	servicios, DI, configuración y logging
Datos	¿Cómo persiste la API?	EF Core, SQLite, migraciones, consultas y concurrencia
Operación	¿Cómo se mantiene fiable?	errores, seguridad, OpenAPI, pruebas, métricas y despliegue
Alternativas	¿REST es siempre suficiente?	RPC, webhooks, SignalR, WebSockets y gRPC

Estructura propuesta

Carpetas orientativas	
1	Biblioteca.Api/
2	├─ Endpoints/
3	├─ Controllers/
4	├─ Contracts/
5	├─ Domain/
6	├─ Services/
7	├─ Infrastructure/
8	└─ BibliotecaDbContext.cs
9	├─ Hubs/
10	├─ Program.cs
11	└─ appsettings.json

Un solo contrato, varias interfaces

Minimal APIs y Controllers son dos formas de declarar endpoints sobre la misma plataforma. No deberían duplicar reglas: ambos pueden llamar a los mismos servicios de aplicación y al mismo DbContext.

2. Qué es ASP.NET Core y qué problema resuelve

ASP.NET Core es la plataforma web multiplataforma de .NET. Proporciona servidor HTTP, hosting, configuración, logging, inyección de dependencias, middleware, routing, seguridad y modelos de programación para HTML, API y tiempo real. Evita que cada aplicación tenga que construir por separado toda esa infraestructura transversal.

El problema antes del endpoint

- Aceptar conexiones HTTP y HTTPS de forma eficiente y cerrar la aplicación correctamente.
- Convertir una URL y un verbo en código ejecutable mediante routing.
- Deserializar entrada, serializar salida y negociar formatos.
- Autenticar, autorizar, limitar tráfico y aplicar CORS.
- Inyectar servicios con ciclos de vida controlados.
- Gestionar configuración por entorno, secretos y observabilidad.
- Responder de manera uniforme ante validaciones, ausencias y excepciones.

Qué no es

No es únicamente una biblioteca para devolver JSON ni un generador de CRUD. Tampoco obliga a REST: sobre el mismo host se pueden publicar páginas, controladores, Minimal APIs, hubs de SignalR, WebSockets y servicios gRPC.

Concepto	Papel
.NET	runtime, bibliotecas base, compilador y herramientas
ASP.NET Core	plataforma web y canalización HTTP
Entity Framework Core	mapeo objeto-relacional y acceso a datos
Kestrel	servidor web multiplataforma incluido
OpenAPI	descripción legible por máquinas del contrato HTTP

3. HTTP como lenguaje de la API

Anatomía de la petición

```
Petición HTTP conceptual
1  POST /api/libros HTTP/1.1
2  Host: localhost:7123
3  Content-Type: application/json
4  Accept: application/json
5  Authorization: Bearer eyJ...
6
7  {
8    "titulo": "La invención de Morel",
9    "isbn": "9788420674271",
10   "anio": 1940
11 }
```

El método expresa la intención; la ruta identifica el recurso; las cabeceras transportan metadatos y el cuerpo contiene una representación. La respuesta repite la misma estructura con estado, cabeceras y cuerpo.

Propiedades que condicionan el diseño

Propiedad	Significado	Ejemplo
Seguro	no solicita un cambio de estado	GET, HEAD
Idempotente	repetir produce el mismo efecto deseado	PUT, DELETE
Cacheable	la respuesta puede reutilizarse si las reglas lo permiten	GET con Cache-Control
Sin estado	cada petición contiene lo necesario para interpretarla	token y parámetros en la petición

Idempotencia no significa respuesta idéntica

Repetir DELETE puede devolver primero 204 y después 404; lo importante es que el estado final siga siendo 'recurso ausente'. Para POST críticos puede admitirse una clave de idempotencia.

4. Crear el proyecto y dominar las herramientas

Plantillas

Terminal

```
1 dotnet --info
2 dotnet new web -n Biblioteca.Api
3 dotnet new webapi -n Biblioteca.Controllers --use-controllers
4 dotnet new list web
5 dotnet restore
6 dotnet build
7 dotnet watch
```

La plantilla **web** parte de un host vacío apropiado para Minimal APIs. La plantilla **webapi** prepara una API y puede generar controladores. En ambos casos el proyecto usa **Microsoft.NET.Sdk.Web** y el destino **net10.0**.

Biblioteca.Api.csproj

```
1 <Project Sdk="Microsoft.NET.Sdk.Web">
2   <PropertyGroup>
3     <TargetFramework>net10.0</TargetFramework>
4     <Nullable>enable</Nullable>
5     <ImplicitUsings>enable</ImplicitUsings>
6   </PropertyGroup>
7 </Project>
```

Comprobar sin depender del navegador

Pruebas manuales

```
1 curl -i https://localhost:7123/health
2 curl -i -H "Accept: application/json" https://localhost:7123/api/libros
3 curl -i -X POST https://localhost:7123/api/libros -H "Content-Type: application/json" -d '{"titulo":"Nada","isbn":"9788423342368","anio":1945}'
```

Actividades propuestas

1. Crea un proyecto vacío net10.0, añade /health y comprueba estado, cabeceras y cuerpo con curl.
2. Ejecuta con dotnet watch y observa qué cambios pueden aplicarse sin reiniciar manualmente.

5. Host, Kestrel, middleware, routing y endpoints

WebApplicationBuilder reúne configuración, logging, servicios y servidor. Tras **Build**, **WebApplication** representa la aplicación ejecutable. Cada petición atraviesa middleware ordenado y, si el routing encuentra coincidencia, llega a un endpoint.

Program.cs

```
1 var builder = WebApplication.CreateBuilder(args);
2
3 builder.Services.AddProblemDetails();
4 builder.Services.AddOpenApi();
5
6 var app = builder.Build();
7
8 app.UseExceptionHandler();
9 app.UseHttpsRedirection();
10
11 if (app.Environment.IsDevelopment())
12     app.MapOpenApi();
13
14 app.MapGet("/health", () => TypedResults.Ok(new { status = "ok" }));
15 app.Run();
```

Middleware o filtro

Mecanismo	Alcance	Usos
Middleware	toda la canalización o una rama	errores, correlación, seguridad, compresión
Endpoint filter	Minimal APIs seleccionadas	validación adicional, auditoría, reglas de endpoint
Filtro MVC	controladores y acciones	autorización, acción, resultado y excepciones MVC

El orden es comportamiento

Un middleware solo envuelve lo que aparece después. La gestión global de excepciones debe situarse antes del código que puede lanzar; CORS, autenticación y autorización también requieren un orden coherente.

6. Primera API con Minimal APIs

Una Minimal API relaciona directamente un patrón de ruta con un route handler. Es una buena opción para servicios pequeños, microservicios, prototipos y API donde conviene reducir ceremonia. Al crecer, se organiza mediante grupos, métodos de extensión y servicios.

Program.cs

```
1  var builder = WebApplication.CreateBuilder(args);
2  var app = builder.Build();
3
4  var libros = new List<LibroDto>
5  {
6      new(1, "Nada", "Carmen Laforet", 1945)
7  };
8
9  app.MapGet("/api/libros", () => TypedResults.Ok(libros));
10
11 app.MapGet("/api/libros/{id:int}", Results<Ok<LibroDto>, NotFound> (int id) =>
12 {
13     var libro = libros.SingleOrDefault(x => x.Id == id);
14     return libro is null ? TypedResults.NotFound() : TypedResults.Ok(libro);
15 });
16
17 app.Run();
18
19 public record LibroDto(int Id, string Titulo, string Autor, int Anio);
```

Por qué TypedResults

TypedResults conserva información de tipos y facilita metadatos OpenAPI y pruebas. Cuando un handler admite varias respuestas, **Results<T1,T2>** expresa explícitamente el conjunto permitido.

7. Routing y model binding en Minimal APIs

Fuentes de datos

Fuente	Ejemplo	Uso
Ruta	/libros/{id:int}	identificador obligatorio
Query string	?pagina=2&tamano=20	filtros y opciones
Cabecera	If-Match: valor	metadatos de protocolo
Cuerpo	JSON de LibroCrear	representación compleja
Servicio	BibliotecaDbContext	dependencia resuelta por DI

C#

```
1 app.MapGet("/api/libros/{id:int}", async (
2     [FromRoute] int id,
3     [FromQuery] bool incluirPrestamos,
4     [FromHeader(Name = "X-Correlation-Id")] string? correlacion,
5     BibliotecaDbContext db,
6     CancellationToken ct) =>
7 {
8     var libro = await db.Libros
9         .AsNoTracking()
10        .SingleOrDefaultAsync(x => x.Id == id, ct);
11
12    return libro is null
13        ? Results.NotFound()
14        : Results.Ok(libro);
15 });
```

Restricciones de ruta

- Usa **{id:int}**, **{codigo:guid}** o **{slug:minlength(3)}** para desambiguar rutas.
- Una restricción decide si la ruta coincide; no sustituye la validación de negocio.
- No conviertas cada filtro en un segmento. La query string representa mejor opciones combinables.

8. Organizar Minimal APIs que crecen

Endpoints/LibrosEndpoints.cs

```
1 public static class LibrosEndpoints
2 {
3     public static RouteGroupBuilder MapLibros(this IEndpointRouteBuilder app)
4     {
5         var group = app.MapGroup("/api/libros")
6             .WithTags("Libros");
7
8         group.MapGet("/", ObtenerTodos).WithName("ObtenerLibros");
9         group.MapGet("/{id:int}", ObtenerPorId).WithName("ObtenerLibro");
10        group.MapPost("/", Crear).WithName("CrearLibro");
11        group.MapPut("/{id:int}", Sustituir).WithName("SustituirLibro");
12        group.MapDelete("/{id:int}", Eliminar).WithName("EliminarLibro");
13        return group;
14    }
15 }
```

Los grupos comparten prefijo, etiquetas, autorización, filtros o límites de tráfico. Los handlers pueden ser métodos estáticos privados. Cuando contienen demasiada lógica, la solución no es crear lambdas más grandes: es trasladar reglas a servicios de aplicación.

Filtros de endpoint

CSHARP

```
1 public sealed class CorrelationFilter : IEndpointFilter
2 {
3     public async ValueTask<object?> InvokeAsync(
4         EndpointFilterInvocationContext context,
5         EndpointFilterDelegate next)
6     {
7         var response = context.HttpContext.Response;
8         response.Headers["X-Correlation-Id"] =
9             context.HttpContext.TraceIdentifier;
10        return await next(context);
11    }
12 }
```

Criterio de tamaño

Program.cs debe leerse como un mapa de composición. Si contiene consultas, reglas, validaciones y transformaciones de todos los recursos, la API es mínima solo en el nombre.

9. Crear una API con Controllers

Los controladores agrupan acciones y ofrecen convenciones, filtros MVC, model binding amplio y una estructura familiar para equipos grandes. Heredan normalmente de **ControllerBase**; **Controller** añade soporte para vistas y no es necesario en una API.

Program.cs

```
1 var builder = WebApplication.CreateBuilder(args);
2 builder.Services.AddControllers();
3 builder.Services.AddOpenApi();
4
5 var app = builder.Build();
6 app.UseHttpsRedirection();
7 app.MapControllers();
8 app.Run();
```

Controllers/LibrosController.cs

```
1 [ApiController]
2 [Route("api/[controller]")]
3 public sealed class LibrosController : ControllerBase
4 {
5     private readonly IServicioLibros _servicio;
6
7     public LibrosController(IServicioLibros servicio) => _servicio = servicio;
8
9     [HttpGet("{id:int}")]
10    [ProducesResponseType<LibroDetalleDto>(StatusCodes.Status200OK)]
11    [ProducesResponseType(StatusCodes.Status404NotFound)]
12    public async Task<ActionResult<LibroDetalleDto>> Obtener(
13        int id, CancellationToken ct)
14    {
15        var libro = await _servicio.ObtenerAsync(id, ct);
16        return libro is null ? NotFound() : Ok(libro);
17    }
18 }
```

[ApiController] activa comportamiento orientado a API, incluida la respuesta automática ante ModelState inválido. La ruta puede ser literal para mantener contratos estables aunque se renombre la clase.

10. Minimal APIs frente a Controllers

Criterio	Minimal APIs	Controllers
Ceremonia	muy baja	estructura y atributos explícitos
Organización	grupos y extensiones	clases, acciones y convenciones
Validación .NET 10	AddValidation y filtro generado	ModelState con ApiController
Resultados	TypedResults y Results	ActionResult y IActionResult
Filtros	endpoint filters	pipeline de filtros MVC
Adecuado para	API focalizadas y servicios pequeños	equipos, dominios amplios y convenciones MVC

La documentación actual recomienda comenzar proyectos HTTP nuevos con Minimal APIs, pero Controllers sigue siendo una opción plenamente soportada. La decisión debe basarse en complejidad, experiencia del equipo, convenciones requeridas y claridad, no en una supuesta diferencia de categoría profesional.

Se pueden combinar

Una aplicación puede publicar endpoints mínimos para health checks o webhooks y controladores para un dominio amplio. Lo importante es que compartan contratos y servicios coherentes.

11. Diseñar recursos y rutas REST

REST modela recursos y sus representaciones sobre la semántica de HTTP. Una URL identifica una cosa; el método expresa la operación. El servidor puede implementar internamente servicios, comandos y entidades, pero el contrato externo debe ser estable y comprensible para el cliente.

Evitar	Preferir	Motivo
GET /obtenerLibros	GET /api/libros	el verbo ya expresa consultar
POST /crearLibro	POST /api/libros	la colección crea un miembro
POST /libros/7/borrar	DELETE /api/libros/7	usa semántica HTTP
GET /prestamosDelLibro?id=7	GET /api/libros/7/prestamos	relación navegable
/api/Libro	/api/libros	nombres consistentes y orientados a recursos

Relaciones y acciones

No todas las operaciones encajan en CRUD puro. Una transición de negocio puede exponerse como subrecurso: **POST /prestamos/42/devoluciones** crea el hecho de devolución. Si el dominio es esencialmente un catálogo de comandos, RPC o gRPC quizá sea más honesto que forzar una apariencia REST.

Reglas prácticas

- Usa sustantivos estables y una convención uniforme para pluralización y mayúsculas.
- No reflejes tablas ni estructura interna de clases de forma automática.
- Mantén poca profundidad; más de dos relaciones anidadas suelen indicar una ruta frágil.
- El identificador pertenece a la ruta; filtros, orden, página y proyección suelen ir en query.

12. Métodos HTTP y semántica correcta

Método	Uso	Éxito habitual	Idempotente
GET	obtener colección o miembro	200	sí
POST	crear o ejecutar operación no idempotente	201 o 202	no
PUT	sustituir una representación en URI conocida	200 o 204	sí
PATCH	modificar parcialmente	200 o 204	depende del documento
DELETE	eliminar	204	sí
HEAD	metadatos sin cuerpo	200	sí
OPTIONS	capacidades o preflight CORS	204	sí

PUT no es un PATCH grande

PUT comunica reemplazo completo y repetible. PATCH describe cambios parciales; JSON Patch permite operaciones explícitas, pero exige validar rutas y autorización de cada propiedad. Para actualizaciones sencillas también puede definirse un DTO parcial específico, documentando con precisión el significado de ausente y null.

Creación correcta

CSHARP

```
1 app.MapPost("/api/libros", async Task<CreatedAtRoute<LibroDto>> (
2     LibroCrearDto entrada,
3     BibliotecaDbContext db,
4     CancellationToken ct) =>
5 {
6     var entidad = new Libro { Titulo = entrada.Titulo, Isbn = entrada.Isbn };
7     db.Libros.Add(entidad);
8     await db.SaveChangesAsync(ct);
9     return TypedResults.CreatedAtRoute(
10         new LibroDto(entidad.Id, entidad.Titulo, entidad.Isbn),
11         "ObtenerLibro",
12         new { id = entidad.Id });
13 };
```

13. Códigos de estado y cabeceras

Estado	Cuándo devolverlo	Error típico
200 OK	consulta o actualización con cuerpo	devolverlo para todo
201 Created	recurso creado; incluir Location	omitir la URI creada
202 Accepted	trabajo aceptado pero aún no terminado	simular que ya terminó
204 No Content	éxito sin cuerpo	añadir JSON pese a 204
400 Bad Request	sintaxis, binding o validación de forma	usarlo para ausencia
401 Unauthorized	falta autenticación válida	confundirlo con 403
403 Forbidden	identidad válida sin permiso	ocultar siempre como 401
404 Not Found	recurso no existe o no debe revelarse	responder null con 200
409 Conflict	conflicto con estado actual	tratarlo como error interno
412 Precondition Failed	falló If-Match u otra precondición	ignorar concurrencia
422 Unprocessable Content	semántica inválida bien formada	criterio inconsistente
500 Internal Server Error	fallo inesperado	exponer la excepción

Cabeceras útiles

- **Location** apunta al recurso creado o al estado de una operación asíncrona.
- **ETag** e **If-Match** permiten concurrencia optimista HTTP.
- **Cache-Control**, **Last-Modified** e **If-None-Match** controlan caché condicional.
- **Retry-After** orienta al cliente ante 429 o indisponibilidad temporal.
- **Content-Type** describe la respuesta; **Accept** expresa preferencias del cliente.

14. DTO, contratos y mapeo

Una entidad de EF Core representa persistencia y comportamiento interno; un DTO representa el contrato externo. Devolver entidades directamente acopla esquema y API, puede revelar propiedades, crear ciclos JSON y permitir sobrepublicación de campos.

Contracts/Libros.cs

```
1 public sealed record LibroCrearDto(
2     [property: Required, StringLength(160, MinimumLength = 2)] string Titulo,
3     [property: Required, RegularExpression(@"^[0-9]{13}$")] string Isbn,
4     [property: Range(1450, 2100)] int Anio);
5
6 public sealed record LibroResumenDto(
7     int Id, string Titulo, string Autor, int Anio);
8
9 public sealed record LibroDetalleDto(
10    int Id, string Titulo, string Autor, string Isbn,
11    int Anio, IReadOnlyList<PrestamoDto> Prestamos);
```

Proyección en la consulta

CSHARP

```
1 var resultado = await db.Libros
2     .AsNoTracking()
3     .Where(x => x.Titulo.Contains(texto))
4     .OrderBy(x => x.Titulo)
5     .Select(x => new LibroResumenDto(
6         x.Id, x.Titulo, x.Autor.Nombre, x.Anio))
7     .ToListAsync(ct);
```

Contrato primero

Diseña el DTO según lo que necesita el cliente y proyecta desde la consulta. No cargues un grafo completo con Include para descartarlo después.

15. Validación en .NET 10

Validar evita que datos sintácticamente correctos entren en un estado inválido. Conviene separar forma —requerido, longitud, rango— de reglas de negocio que consultan estado o coordinan varios agregados.

Minimal APIs

Validación integrada en .NET 10

```
1 var builder = WebApplication.CreateBuilder(args);
2 builder.Services.AddValidation();
3 builder.Services.AddProblemDetails();
4
5 var app = builder.Build();
6
7 app.MapPost("/api/libros", (LibroCrearDto entrada) =>
8     TypedResults.Ok(entrada));
```

En .NET 10, **AddValidation** habilita validación integrada de parámetros, tipos y propiedades en Minimal APIs. Los atributos de DataAnnotations pueden aplicarse a ruta, query, cabecera o cuerpo. Un filtro generado ejecuta la validación antes del handler y la respuesta se integra con *IProblemDetailsService*.

Controllers

Con **[ApiController]**, un *ModelState* inválido produce automáticamente una respuesta 400 con detalles de validación. Sin ese atributo hay que comprobar **ModelState.IsValid**. Las reglas de negocio siguen perteneciendo al servicio y pueden generar 409 o 422 según el contrato adoptado.

Capa	Ejemplos	Respuesta
Binding	JSON mal formado, int no convertible	400
DataAnnotations	Required, Range, StringLength	400 con ValidationProblemDetails
Negocio	ISBN duplicado, préstamo imposible	409 o 422 documentado
Persistencia	restricción única o concurrencia	traducir a 409/412; no filtrar SQL

16. Errores coherentes con ProblemDetails

RFC 9457 define un formato de problema para respuestas HTTP. ASP.NET Core representa ese formato con **ProblemDetails** y **ValidationProblemDetails**. El objetivo es que clientes y observabilidad reciban una estructura predecible sin revelar trazas ni detalles internos.

Program.cs

```
1 builder.Services.AddProblemDetails(options =>
2 {
3     options.CustomizeProblemDetails = context =>
4     {
5         context.ProblemDetails.Extensions["traceId"] =
6             context.HttpContext.TraceIdentifier;
7     };
8 });
9
10 var app = builder.Build();
11 app.UseExceptionHandler();
12 app.UseStatusCodePages();
```

CSHARP

```
1 return TypedResults.Problem(
2     statusCode: StatusCodes.Status409Conflict,
3     title: "El ISBN ya existe",
4     detail: "Ya hay un libro registrado con el ISBN indicado.",
5     type: "https://ejemplo.test/problemas/isbn-duplicado",
6     extensions: new Dictionary<string, object?>
7     {
8         ["codigo"] = "ISBN_DUPLICADO"
9     });
```

Política de excepciones

- Captura solo cuando puedas recuperar, traducir o añadir contexto útil.
- Una excepción inesperada llega al manejador global y produce 500 genérico.
- No uses excepciones para una ausencia esperada en cada consulta.
- Registra el detalle interno con `traceId`; devuelve al cliente un problema seguro.
- No devuelvas stack trace, SQL, rutas locales, secretos ni mensajes del proveedor.

17. Inyección de dependencias y servicios

La inyección de dependencias separa la composición del uso. El consumidor declara lo que necesita y el contenedor construye el grafo. Esto reduce acoplamiento y permite sustituir infraestructura durante pruebas.

Ciclo	Instancia	Adecuado	Riesgo
Transient	cada resolución	servicio ligero y sin estado	coste si es pesado
Scoped	una por petición	DbContext y caso de uso	capturarlo desde singleton
Singleton	una por proceso	estado inmutable y thread-safe	estado compartido no seguro

CSHARP

```
1 builder.Services.AddScoped<IServicioLibros, ServicioLibros>();
2 builder.Services.AddSingleton<IClock, SystemClock>();
3 builder.Services.AddHttpClient<IClienteEditorial, ClienteEditorial>(client =>
4 {
5     client.BaseAddress = new Uri("https://editorial.example/");
6     client.Timeout = TimeSpan.FromSeconds(10);
7 });
```

CSHARP

```
1 public sealed class ServicioLibros(
2     BibliotecaDbContext db,
3     ILogger<ServicioLibros> logger,
4     IClock clock) : IServicioLibros
5 {
6     public async Task<LibroDetalleDto?> ObtenerAsync(
7         int id, CancellationToken ct)
8     {
9         logger.LogInformation("Consultando libro {LibroId}", id);
10        return await db.Libros.AsNoTracking()
11            .Where(x => x.Id == id)
12            .Select(x => new LibroDetalleDto(/* proyección */))
13            .SingleOrDefaultAsync(ct);
14    }
15 }
```

No uses IServiceProvider como localizador

Resolver servicios manualmente dentro de la lógica oculta dependencias y dificulta las pruebas. La resolución explícita se reserva para composición o situaciones de infraestructura justificadas.

18. Configuración, Options y secretos

appsettings.json

```
1 {
2   "ConnectionStrings": {
3     "Biblioteca": "Data Source=biblioteca.db"
4   },
5   "Catalogo": {
6     "BaseUrl": "https://catalogo.example/",
7     "TimeoutSeconds": 10
8   }
9 }
```

CSHARP

```
1 public sealed class CatalogoOptions
2 {
3     public const string SectionName = "Catalogo";
4     [Required, Url] public string BaseUrl { get; init; } = "";
5     [Range(1, 60)] public int TimeoutSeconds { get; init; } = 10;
6 }
7
8 builder.Services.AddOptions<CatalogoOptions>()
9     .BindConfiguration(CatalogoOptions.SectionName)
10    .ValidateDataAnnotations()
11    .ValidateOnStart();
```

Interfaz	Comportamiento	Uso
IOptions	valor estable	configuración singleton
IOptionsSnapshot	se recalcula por petición	servicios scoped
IOptionsMonitor	observa cambios	singleton y callbacks

Prioridad y secretos

La configuración se combina desde appsettings, fichero por entorno, secretos de desarrollo, variables de entorno y argumentos, entre otras fuentes. Una fuente posterior puede sobrescribir otra. Las claves reales, cadenas de producción y certificados no deben confirmarse en Git; se inyectan desde un almacén de secretos o el entorno de despliegue.

BASH

```
1 dotnet user-secrets init
2 dotnet user-secrets set "ApiKeys:Catalogo" "valor-local"
3
4 # Variable jerárquica en despliegue
5 ConnectionStrings__Biblioteca="Data Source=/data/biblioteca.db"
```

19. Logging, trazas y cancelación

Logging estructurado

CSHARP

```
1 logger.LogInformation(  
2     "Préstamo {PrestamoId} devuelto por {UsuarioId} en {DuracionMs} ms",  
3     prestamo.Id, usuarioId, duracion.TotalMilliseconds);
```

Las plantillas conservan propiedades consultables. No interpolas toda la frase ni registres cuerpos, tokens o datos personales por defecto. Usa niveles Trace, Debug, Information, Warning, Error y Critical de forma coherente.

CancellationToken

CSHARP

```
1 app.MapGet("/api/libros", async (  
2     BibliotecaDbContext db,  
3     CancellationToken ct) =>  
4 {  
5     var libros = await db.Libros.AsNoTracking()  
6         .OrderBy(x => x.Titulo)  
7         .Select(x => new { x.Id, x.Titulo })  
8         .ToListAsync(ct);  
9     return TypedResults.Ok(libros);  
10 });
```

El token suele representar **RequestAborted**. Debe propagarse a EF Core, HttpClient y operaciones asíncronas para liberar trabajo cuando el cliente se desconecta o expira el tiempo. No lo sustituyas por `CancellationToken.None`.

Correlación

`TraceIdentifier`, `Activity` y cabeceras de correlación permiten unir logs, trazas y llamadas salientes. `OpenTelemetry` puede exportar trazas, métricas y logs a un sistema de observabilidad.

20. Conectar Entity Framework Core 10

Terminal

```
1 dotnet add package Microsoft.EntityFrameworkCore.Sqlite --version 10.0.0
2 dotnet add package Microsoft.EntityFrameworkCore.Design --version 10.0.0
3 dotnet tool install --global dotnet-ef
4 dotnet ef migrations add Inicial
5 dotnet ef database update
```

Infrastructure/BibliotecaDbContext.cs

```
1 public sealed class BibliotecaDbContext(DbContextOptions<BibliotecaDbContext> options)
2     : DbContext(options)
3 {
4     public DbSet<Libro> Libros => Set<Libro>();
5     public DbSet<Autor> Autores => Set<Autor>();
6     public DbSet<Prestamo> Prestamos => Set<Prestamo>();
7
8     protected override void OnModelCreating(ModelBuilder modelBuilder)
9     {
10         modelBuilder.Entity<Libro>()
11             .HasIndex(x => x.Isbn)
12             .IsUnique();
13         modelBuilder.Entity<Libro>()
14             .Property(x => x.Titulo)
15             .HasMaxLength(160);
16     }
17 }
```

Program.cs

```
1 builder.Services.AddDbContext<BibliotecaDbContext>(options =>
2     options.UseSqlite(
3         builder.Configuration.GetConnectionString("Biblioteca")));
```

DbContext es scoped

Una petición obtiene normalmente una instancia. No es thread-safe: no ejecute operaciones paralelas sobre el mismo contexto ni lo guardes en un singleton.

21. CRUD con Minimal APIs y EF Core

Consulta y detalle

CSHARP

```
1 group.MapGet("/", async (BibliotecaDbContext db, CancellationToken ct) =>
2     TypedResults.Ok(await db.Libros.AsNoTracking()
3         .OrderBy(x => x.Titulo)
4         .Select(x => new LibroResumenDto(x.Id, x.Titulo, x.Autor.Nombre, x.Anio))
5         .ToListAsync(ct)));
6
7 group.MapGet("/{id:int}", async Task<Results<Ok<LibroResumenDto>, NotFound>> (
8     int id, BibliotecaDbContext db, CancellationToken ct) =>
9 {
10     var dto = await db.Libros.AsNoTracking()
11         .Where(x => x.Id == id)
12         .Select(x => new LibroResumenDto(x.Id, x.Titulo, x.Autor.Nombre, x.Anio))
13         .SingleOrDefaultAsync(ct);
14     return dto is null ? TypedResults.NotFound() : TypedResults.Ok(dto);
15 };
```

Actualizar y borrar

CSHARP

```
1 group.MapPut("/{id:int}", async (int id, LibroActualizarDto dto,
2     BibliotecaDbContext db, CancellationToken ct) =>
3 {
4     var libro = await db.Libros.FindAsync([id], ct);
5     if (libro is null) return Results.NotFound();
6
7     libro.Titulo = dto.Titulo;
8     libro.Anio = dto.Anio;
9     await db.SaveChangesAsync(ct);
10    return Results.NoContent();
11 });
12
13 group.MapDelete("/{id:int}", async (int id, BibliotecaDbContext db,
14     CancellationToken ct) =>
15 {
16     var filas = await db.Libros.Where(x => x.Id == id).ExecuteDeleteAsync(ct);
17     return filas == 0 ? Results.NotFound() : Results.NoContent();
18 };
```

ExecuteDeleteAsync evita cargar la entidad y ejecuta una eliminación directa. No activa lógica en memoria ni seguimiento; úsalo solo cuando eso coincida con las reglas del caso de uso.

22. CRUD con Controllers y un servicio

Controllers/LibrosController.cs

```
1 [HttpPost]
2 public async Task<ActionResult<LibroDetalleDto>> Crear(
3     LibroCrearDto entrada, CancellationToken ct)
4 {
5     var resultado = await _servicio.CrearAsync(entrada, ct);
6     if (resultado.EsDuplicado)
7         return Conflict(new ProblemDetails
8             {
9                 Title = "El ISBN ya existe",
10                 Status = StatusCodes.Status409Conflict
11             });
12
13     return CreatedAtAction(
14         nameof(Obtener),
15         new { id = resultado.Libro!.Id },
16         resultado.Libro);
17 }
18
19 [HttpDelete("{id:int}")]
20 public async Task<ActionResult> Eliminar(int id, CancellationToken ct)
21 {
22     return await _servicio.EliminarAsync(id, ct)
23         ? NoContent()
24         : NotFound();
25 }
```

El controlador adapta HTTP al caso de uso: extrae entrada, llama al servicio y traduce el resultado a estado y representación. No debe contener consultas complejas ni reglas duplicadas.

Resultado de aplicación

Un servicio puede devolver un tipo resultado con estados como éxito, ausente, duplicado o conflicto. La capa HTTP traduce esos estados. Así, la misma lógica puede usarse desde un controlador, endpoint mínimo, consumidor de mensajes o prueba unitaria.

23. Relaciones, transacciones y migraciones

Relaciones

CSHARP

```
1 modelBuilder.Entity<Libro>()
2     .HasOne(x => x.Autor)
3     .WithMany(x => x.Libros)
4     .HasForeignKey(x => x.AutorId)
5     .OnDelete(DeleteBehavior.Restrict);
6
7 modelBuilder.Entity<Prestamo>()
8     .HasOne(x => x.Libro)
9     .WithMany(x => x.Prestamos)
10    .HasForeignKey(x => x.LibroId);
```

Transacciones

SaveChanges es transaccional para sus cambios. Si un caso de uso necesita varias llamadas a SaveChanges o combina operaciones que deben confirmarse juntas, puede abrir una transacción explícita. No mantengas una transacción mientras esperas una API remota.

CSHARP

```
1 await using var tx = await db.Database.BeginTransactionAsync(ct);
2 prestamo.FechaDevolucion = clock.UtcNow;
3 libro.Disponible = true;
4 await db.SaveChangesAsync(ct);
5 await tx.CommitAsync(ct);
```

Migraciones en equipos

- Cada migración se revisa como código y se prueba sobre una copia de datos representativa.
- No borres una migración ya aplicada en entornos compartidos; crea una corrección.
- Genera scripts idempotentes o un paquete de migración para despliegues controlados.
- Separar despliegue de esquema y aplicación reduce riesgos en cambios incompatibles.

24. Consultas eficientes, paginación y búsqueda

CSHARP

```
1 public sealed record Pagina<T>(  
2     IReadOnlyList<T> Elementos, int PaginaActual,  
3     int Tamano, int Total);  
4  
5 var consulta = db.Libros.AsNoTracking();  
6  
7 if (!string.IsNullOrWhiteSpace(q))  
8     consulta = consulta.Where(x => x.Titulo.Contains(q));  
9  
10 var total = await consulta.CountAsync(ct);  
11 var elementos = await consulta  
12     .OrderBy(x => x.Titulo).ThenBy(x => x.Id)  
13     .Skip((pagina - 1) * tamano)  
14     .Take(tamano)  
15     .Select(x => new LibroResumenDto(x.Id, x.Titulo, x.Autor.Nombre, x.Anio))  
16     .ToListAsync(ct);
```

Reglas de rendimiento

- Limita tamaño con un máximo del servidor; no permitas descargar toda la tabla.
- Ordena de manera determinista antes de Skip y Take.
- Usa AsNoTracking para lecturas que no se actualizarán.
- Proyecta columnas necesarias y revisa el SQL generado.
- Evita N+1; proyecta relaciones o carga de forma explícita y medida.
- Para conjuntos muy grandes, la paginación por cursor evita el coste creciente de Skip.

Contains no siempre escala

El comportamiento depende del proveedor y la intercalación. Búsquedas lingüísticas, por relevancia o con grandes volúmenes pueden necesitar índices específicos o un motor de búsqueda.

25. Concurrencia e idempotencia

Dos clientes pueden leer la misma versión y guardar cambios incompatibles. La concurrencia optimista detecta que la versión cambió y evita que el último escritor silencie al primero.

CSHARP

```
1 public sealed class Libro
2 {
3     public int Id { get; set; }
4     public string Titulo { get; set; } = "";
5     [Timestamp]
6     public byte[] Version { get; set; } = [];
7 }
```

La versión puede exponerse como ETag. El cliente la devuelve en If-Match; si ya no coincide, la API responde 412. EF Core lanza `DbUpdateConcurrencyException` cuando la fila esperada no se actualiza.

Claves de idempotencia

Pagos, reservas o creación desde clientes que reintentan pueden admitir **Idempotency-Key**. El servidor registra clave, identidad, huella de la petición y resultado durante una ventana. Repetir la misma petición devuelve el resultado anterior; reutilizar la clave con otro cuerpo es un conflicto.

No basta un diccionario en memoria

En varias instancias o tras un reinicio, la idempotencia requiere un almacén compartido y una operación atómica. Define caducidad y tratamiento de peticiones en curso.

26. OpenAPI y documentación del contrato

CSHARP

```
1 builder.Services.AddOpenApi();
2
3 var app = builder.Build();
4
5 if (app.Environment.IsDevelopment())
6     app.MapOpenApi();
7
8 app.MapGet("/api/libros/{id:int}", ObtenerLibro)
9     .WithName("ObtenerLibro")
10    .WithSummary("Obtiene un libro por identificador")
11    .Produces<LibroDetalleDto>(StatusCodes.Status200OK)
12    .ProducesProblem(StatusCodes.Status404NotFound);
```

ASP.NET Core genera documentos OpenAPI a partir de endpoints y metadatos. En .NET 10 puede producir OpenAPI 3.1. El documento no sustituye el diseño: describe con precisión lo que la aplicación declara, incluidos esquemas, parámetros, respuestas y seguridad.

Qué documentar

- Finalidad del endpoint y permisos necesarios.
- Formato, restricciones y ejemplos de cada parámetro.
- Todas las respuestas relevantes, también 400, 401, 403, 404, 409, 412, 422 y 429.
- ProblemDetails y códigos de problema estables.
- Paginación, límites, idempotencia y política de versiones.
- No publiques el documento de desarrollo sin decidir su exposición y protección.

27. Versionar y evolucionar una API

Una API evoluciona sin romper clientes cuando los cambios compatibles son aditivos y los incompatibles se planifican. Añadir una propiedad opcional suele ser compatible; renombrar o cambiar tipo, semántica o cardinalidad puede romper deserialización y lógica.

Estrategia	Ventaja	Coste
Ruta /api/v1	visible y sencilla	URI nueva y duplicación de rutas
Cabecera	URI limpia	menos visible y peor prueba manual
Media type	versiona representación	mayor complejidad operativa
Evolución compatible	menos versiones	exige disciplina contractual

Retirada

- Mide qué versiones y campos usan los clientes.
- Anuncia fechas y guía de migración con antelación.
- Mantén pruebas de contrato para versiones soportadas.
- Evita mantener indefinidamente ramas de lógica casi iguales.

28. Seguridad básica de una API

HTTPS protege el transporte, pero una API segura necesita autenticación, autorización, validación, gestión de secretos, límites, actualizaciones y una superficie mínima.

Autenticación y autorización

CSHARP

```
1 builder.Services.AddAuthentication().AddJwtBearer();
2 builder.Services.AddAuthorization(options =>
3 {
4     options.AddPolicy("Bibliotecario", policy =>
5         policy.RequireClaim("permiso", "catalogo.escribir"));
6 });
7
8 app.MapPost("/api/libros", CrearLibro)
9     .RequireAuthorization("Bibliotecario");
```

CORS

CORS es una política del navegador para llamadas entre orígenes; no autentica ni protege clientes no navegador. Configura orígenes, métodos y cabeceras concretos. No combines credenciales con un origen comodín.

CSHARP

```
1 builder.Services.AddCors(options =>
2 {
3     options.AddPolicy("Frontend", policy => policy
4         .WithOrigins("https://app.example")
5         .WithMethods("GET", "POST", "PUT", "DELETE")
6         .WithHeaders("Content-Type", "Authorization"));
7
8 app.UseCors("Frontend");
```

Checklist

- Autoriza por recurso, no solo por ruta o rol global.
- Limita tamaño de cuerpos y subida de archivos.
- Evita asignación masiva mediante DTO de entrada.
- No registres tokens, contraseñas, claves ni datos personales innecesarios.
- Actualiza runtime y dependencias y revisa avisos de seguridad.

29. Rate limiting, caché y resiliencia

CSHARP

```
1 builder.Services.AddRateLimiter(options =>
2 {
3     options.AddFixedWindowLimiter("lecturas", limiter =>
4     {
5         limiter.PermitLimit = 100;
6         limiter.Window = TimeSpan.FromMinutes(1);
7         limiter.QueueLimit = 0;
8     });
9 });
10
11 app.UseRateLimiter();
12 app.MapGet("/api/libros", ObtenerLibros)
13     .RequireRateLimiting("lecturas");
```

Limitar tráfico protege capacidad y equidad; no sustituye autenticación. La partición debería considerar identidad o cliente, evitando depender solo de IP detrás de proxies.

Caché HTTP

Un GET público puede usar ETag, Last-Modified y Cache-Control. Los datos privados requieren directivas cuidadosas y claves que incluyan variaciones relevantes. La caché de salida del servidor y la caché HTTP del cliente resuelven problemas distintos.

Llamadas salientes

Usa `IHttpClientFactory`, timeouts y reintentos solo para operaciones seguras o idempotentes. Añade jitter, circuit breaker cuando proceda y un presupuesto total. Reintentar sin límite multiplica una caída.

30. Pruebas de servicios y endpoints

Pirámide práctica

Nivel	Comprueba	Herramienta
Unidad	reglas y mapeos sin infraestructura	xUnit
Integración	routing, DI, filtros, JSON, EF y seguridad	WebApplicationFactory
Contrato	compatibilidad de OpenAPI o consumidor	snapshot o pruebas de contrato
Extremo a extremo	flujo real desde cliente	Playwright u otro cliente

CSHARP

```
1 public sealed class LibrosApiTests : IClassFixture<WebApplicationFactory<Program>>
2 {
3     private readonly HttpClient _client;
4
5     public LibrosApiTests(WebApplicationFactory<Program> factory)
6     => _client = factory.CreateClient();
7
8     [Fact]
9     public async Task Obtener_id_inexistente_devuelve_404()
10    {
11        var response = await _client.GetAsync("/api/libros/999999");
12        Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
13    }
14 }
```

Base de datos de prueba

SQLite en memoria se aproxima mejor a una base relacional que el proveedor InMemory. Cada prueba debe controlar esquema y datos, evitar compartir estado mutable y comprobar tanto respuesta como efectos persistentes.

Actividades propuestas

1. Prueba 201 y Location al crear un libro válido.
2. Prueba ValidationProblemDetails al omitir título.
3. Prueba 409 ante ISBN duplicado y 404 al borrar un id ausente.
4. Comprueba que una petición sin permiso no modifica la base de datos.

31. REST no es la única opción

REST funciona bien para recursos consultables y mutables mediante HTTP. Otros estilos priorizan comandos, notificaciones, tiempo real, streaming o contratos binarios. Una arquitectura puede combinar varios, siempre que cada frontera tenga una razón operativa.

Opción	Dirección	Fortaleza	Límite
REST	cliente solicita	recursos, caché y ecosistema HTTP	no empuja eventos por sí solo
RPC	cliente invoca	acciones y comandos explícitos	acoplamiento a operaciones
Webhook	servidor llama al suscriptor	notificación asíncrona	entrega al menos una vez
SignalR	bidireccional	tiempo real con abstracción	estado de conexiones y escalado
WebSocket	bidireccional	canal de bajo nivel	protocolo propio y más trabajo
gRPC	cliente invoca/streaming	contrato tipado y eficiencia	compatibilidad de navegador/proxy

32. RPC: operaciones como contrato

RPC modela una llamada a un procedimiento: **reservarLibro**, **renovarPrestamo** o **calcularMulta**. Puede transportarse sobre HTTP con JSON sin ser REST. Es apropiado cuando el dominio es un conjunto de comandos y el valor está en la operación, no en manipular representaciones de recursos.

CSHARP

```
1 app.MapPost("/rpc/renovar-prestamo", async (
2     RenovarPrestamoRequest request,
3     IServicioPrestamos servicio,
4     CancellationToken ct) =>
5 {
6     var resultado = await servicio.RenovarAsync(
7         request.PrestamoId, request.Dias, ct);
8     return resultado.Exito
9         ? Results.Ok(new RenovarPrestamoResponse(resultado.NuevaFecha))
10        : Results.Problem(statusCode: 409, title: resultado.Error);
11 });
```

Diseñarlo bien

- Usa nombres de operación estables, solicitudes y respuestas tipadas.
- Documenta idempotencia, errores y efectos de cada comando.
- No lo disfraces de REST si la semántica real es una llamada.
- Separa transporte RPC de reglas para poder migrar a gRPC o mensajería.

33. Webhooks: notificar eventos a otros sistemas

Un webhook invierte la iniciativa: cuando ocurre un evento, el productor realiza una petición HTTP al receptor. Evita sondeo continuo, pero introduce reintentos, duplicados, orden variable y seguridad de entregas.

Receptor seguro

CSHARP

```
1 app.MapPost("/webhooks/editorial", async (
2     HttpRequest request,
3     IVerificadorWebhook verificador,
4     IColaEventos cola,
5     CancellationToken ct) =>
6 {
7     using var reader = new StreamReader(request.Body);
8     var body = await reader.ReadToEndAsync(ct);
9     var firma = request.Headers["X-Signature"].ToString();
10
11     if (!verificador.EsValida(body, firma))
12         return Results.Unauthorized();
13
14     var evento = JsonSerializer.Deserialize<EventoEditorial>(body);
15     await cola.EncolarAsync(evento!, ct);
16     return Results.Accepted();
17 });
```

Reglas de entrega

1. Verifica firma sobre los bytes originales, secreto, marca temporal y tolerancia anti-replay.
2. Deduplica por identificador de entrega en un almacén compartido.
3. Responde rápido con 2xx y procesa trabajo pesado fuera de la petición.
4. Acepta entrega al menos una vez: el mismo evento puede repetirse.
5. Registra intentos, duración y resultado sin almacenar secretos o cuerpos sensibles.
6. Como productor, usa timeout, backoff, jitter, cola de fallos y posibilidad de reenvío.

34. SignalR: tiempo real de alto nivel

SignalR permite que el servidor envíe datos a clientes conectados. Gestiona conexiones, hubs, grupos, serialización y elección de transporte. Es apropiado para chats, paneles, colaboración, notificaciones y progreso en vivo.

CSHARP

```
1 builder.Services.AddSignalR();
2
3 var app = builder.Build();
4 app.MapHub<BibliotecaHub>("/hubs/biblioteca");
5
6 public sealed class BibliotecaHub : Hub
7 {
8     public Task UnirseALibro(int libroId) =>
9         Groups.AddToGroupAsync(Context.ConnectionId, $"libro-{libroId}");
10 }
```

CSHARP

```
1 public sealed class NotificadorSignalR(
2     IHubContext<BibliotecaHub> hub) : INotificador
3 {
4     public Task LibroActualizadoAsync(int id, LibroResumenDto dto, CancellationToken ct) =>
5         hub.Clients.Group($"libro-{id}")
6             .SendAsync("LibroActualizado", dto, ct);
7 }
```

Diseño operativo

- Autoriza el hub y cada operación sensible; una conexión autenticada no autoriza todo.
- Mantén mensajes pequeños y limita frecuencia y tamaño.
- No guardes el estado duradero únicamente en el hub.
- Al escalar horizontalmente usa un servicio administrado o backplane compatible.
- El cliente debe reconectar, resincronizar estado y tolerar eventos perdidos.

35. WebSockets: canal bidireccional de bajo nivel

WebSocket mantiene un canal dúplex sobre una conexión. ASP.NET Core permite aceptarlo directamente, pero la aplicación debe definir framing lógico, mensajes, autorización, keepalive, reconexión, presión de retorno y límites. SignalR resuelve gran parte de este trabajo.

CSHARP

```
1  app.UseWebSockets();
2
3  app.Map("/ws", async context =>
4  {
5      if (!context.WebSockets.IsWebSocketRequest)
6      {
7          context.Response.StatusCode = StatusCodes.Status400BadRequest;
8          return;
9      }
10
11     using var socket = await context.WebSockets.AcceptWebSocketAsync();
12     var buffer = new byte[4096];
13     var result = await socket.ReceiveAsync(buffer, context.RequestAborted);
14
15     await socket.SendAsync(
16         buffer.AsMemory(0, result.Count),
17         result.MessageType,
18         result.EndOfMessage,
19         context.RequestAborted);
20 });
```

Cuándo bajar de nivel

Usa WebSockets directos cuando necesitas un protocolo propio y control fino. Para la mayoría de aplicaciones web en tiempo real, SignalR ofrece una abstracción más productiva y robusta.

36. gRPC: contratos tipados y streaming

gRPC utiliza contratos Protocol Buffers, generación de clientes y HTTP/2. En ASP.NET Core se integra con DI, logging, autenticación y autorización. Es especialmente útil entre servicios controlados, para baja latencia, mensajes compactos y streaming.

Protos/catalogo.proto

```
1 syntax = "proto3";
2 option csharp_namespace = "Biblioteca.Grpc";
3 service Catalogo {
4     rpc ObtenerLibro (ObtenerLibroRequest) returns (LibroReply);
5     rpc SeguirCambios (SeguirCambiosRequest) returns (stream LibroReply);
6 }
7 message ObtenerLibroRequest { int32 id = 1; }
8 message SeguirCambiosRequest { int32 desde_id = 1; }
9 message LibroReply { int32 id = 1; string titulo = 2; string autor = 3; }
```

CSHARP

```
1 builder.Services.AddGrpc();
2 var app = builder.Build();
3 app.MapGrpcService<CatalogoService>();
4 public sealed class CatalogoService(IServicioLibros servicio) : Catalogo.CatalogoBase
5 {
6     public override async Task<LibroReply> ObtenerLibro(ObtenerLibroRequest request,
7         ServerCallContext context)
8     {
9         var libro = await servicio.ObtenerAsync(request.Id, context.CancellationToken);
10        return libro is null
11            ? throw new RpcException(new Status(StatusCode.NotFound, "No encontrado"))
12            : new LibroReply { Id = libro.Id, Titulo = libro.Titulo };
13    }
14 }
```

Tipos de llamada

- Unary: una petición y una respuesta.
- Server streaming: una petición y secuencia de respuestas.
- Client streaming: secuencia de peticiones y una respuesta.
- Bidirectional streaming: ambas secuencias avanzan de forma independiente.

Los navegadores no llaman directamente a gRPC HTTP/2 estándar; gRPC-Web y la transcodificación JSON cubren algunos escenarios con compromisos. Los números de campo protobuf no se reutilizan al evolucionar contratos.

37. Elegir REST, RPC, webhooks, SignalR, WebSockets o gRPC

Necesidad principal	Elección inicial	Pregunta de control
CRUD y consultas públicas	REST	¿el dominio se expresa como recursos?
Comandos de negocio	RPC o gRPC	¿la operación es más estable que la representación?
Avisar a un sistema externo	webhook	¿tolera reintentos y duplicados?
Actualizar navegador en vivo	SignalR	¿necesita grupos, reconexión y fallback?
Protocolo dúplex propio	WebSocket	¿se justifica implementar el protocolo?
Servicio a servicio tipado	gRPC	¿infraestructura y clientes soportan HTTP/2?

La elección no es exclusiva. Una API REST puede iniciar un trabajo, devolver 202 con Location, publicar progreso por SignalR y notificar su finalización por webhook. Internamente, dos servicios pueden comunicarse por gRPC.

Primero los requisitos de calidad

Latencia, volumen, topología, compatibilidad, observabilidad, seguridad, reintentos y experiencia del equipo importan tanto como la forma del código.

38. Publicación, salud y operación

CSHARP

```
1 builder.Services.AddHealthChecks()
2     .AddDbContextCheck<BibliotecaDbContext>();
3
4 app.MapHealthChecks("/health/live", new HealthCheckOptions
5 {
6     Predicate = _ => false
7 });
8 app.MapHealthChecks("/health/ready");
```

Liveness y readiness

Liveness indica si el proceso debe reiniciarse; no debe depender de cada sistema remoto. Readiness indica si la instancia puede recibir tráfico y puede comprobar dependencias críticas. Un fallo de una API opcional no siempre debe sacar la aplicación del balanceador.

Publicar

BASH

```
1 dotnet publish -c Release -o ./publish
2 dotnet Biblioteca.Api.dll --urls http://0.0.0.0:8080
```

- Termina TLS en Kestrel o proxy inverso y configura correctamente cabeceras reenviadas.
- Ejecuta con identidad de mínimos privilegios y sistema de archivos restringido.
- No uses launchSettings.json como configuración de producción.
- Aplica migraciones mediante un proceso controlado, no desde cada réplica simultáneamente.
- Define timeout de apagado y permite cierre ordenado de peticiones y consumidores.
- Monitoriza latencia, tasa de error, saturación, tráfico, salud y dependencias.

39. Proyecto integrador: Biblioteca API

Requisitos

1. Publica colecciones y detalle de autores, libros y préstamos.
2. Implementa lecturas con Minimal APIs y administración con Controllers para comparar enfoques.
3. Usa DTO separados, validación .NET 10 y ProblemDetails uniforme.
4. Persiste con EF Core y SQLite, migraciones, restricciones únicas y concurrencia.
5. Añade búsqueda, orden determinista, paginación y CancellationToken.
6. Protege escritura con una política de autorización y limita tráfico de lectura.
7. Genera OpenAPI y documenta todas las respuestas.
8. Publica LibroActualizado por SignalR y recibe un webhook firmado de una editorial.
9. Añade un servicio gRPC interno para consultar inventario.
10. Escribe pruebas unitarias, de integración y de contrato.

Criterios de aceptación

Área	Evidencia
HTTP	rutas de recursos, verbos y estados coherentes
Errores	ProblemDetails seguro con traceId y códigos estables
Datos	migración reproducible, consultas proyectadas y ausencia de N+1
Seguridad	HTTPS, autorización, secretos externos, CORS mínimo y límites
Fiabilidad	cancelación, idempotencia o concurrencia donde corresponda
Pruebas	casos de éxito, validación, ausencia, conflicto y permiso
Operación	health checks, logs estructurados y guía de despliegue

40. Diagnóstico de errores frecuentes

Síntoma	Causa probable	Comprobación
404 en todas las acciones	falta MapControllers o ruta distinta	lista endpoints y revisa atributos
415 Unsupported Media Type	Content-Type no admitido	envía application/json
400 antes del handler	binding o validación automática	lee ValidationProblemDetails
JSON vacío o propiedades omitidas	DTO o serialización	revisa nombres y contrato
DbContext disposed	ciclo de vida incorrecto	no guardes scoped en singleton
Segunda operación EF falla	uso paralelo del mismo DbContext	espera cada operación
CORS solo falla en navegador	política de origen	inspecciona preflight OPTIONS
401 inesperado	esquema, token o audiencia	revisa cabecera y logs seguros
500 revela detalles	manejador de errores ausente	activa ExceptionHandler y ProblemDetails
Webhook duplicado	reintento normal	deduplica delivery id

Método de diagnóstico

1. Reproduce con una petición mínima y conserva método, URL, cabeceras, cuerpo y respuesta.
2. Busca traceId en logs y confirma qué endpoint seleccionó routing.
3. Distingue fallo de transporte, autenticación, binding, validación, negocio o persistencia.
4. Activa logging más detallado solo de forma temporal y sin datos sensibles.
5. Añade una prueba que falle antes de corregir el defecto.

41. Checklist de calidad

Contrato HTTP

- Rutas orientadas a recursos, métodos y estados con semántica correcta.
- DTO de entrada y salida separados de entidades.
- Paginación, límites, orden y filtros documentados.
- 201 incluye Location; errores usan ProblemDetails.
- Cambios incompatibles tienen estrategia de versión y retirada.

Código y datos

- Handlers o acciones delgados; reglas en servicios comprobables.
- Ciclos de vida DI correctos y ninguna dependencia scoped en singleton.
- Configuración tipada validada al arrancar y secretos fuera del repositorio.
- Consultas asíncronas, CancellationToken, proyección y AsNoTracking donde procede.
- Restricciones de base de datos respaldan invariantes críticas.

Operación

- Autenticación, autorización por recurso, CORS mínimo y rate limiting.
- Logs estructurados, trazas, métricas y health checks útiles.
- Pruebas de unidad, integración, contrato y flujos críticos.
- Timeouts, reintentos seguros, idempotencia y cierre ordenado.
- Documentación OpenAPI y procedimiento de despliegue reproducible.

42. Glosario

Término	Definición
Endpoint	combinación de ruta, método, metadatos y código ejecutable
Middleware	componente que procesa una petición antes o después del siguiente
Binding	conversión de datos HTTP a parámetros u objetos .NET
DTO	objeto que define datos en una frontera
REST	estilo de arquitectura centrado en recursos y semántica HTTP
Idempotencia	propiedad por la que repetir una operación conserva el efecto deseado
ProblemDetails	representación estándar de un problema HTTP
OpenAPI	descripción formal de una API HTTP
Webhook	notificación HTTP iniciada por el productor de un evento
Hub	abstracción de SignalR para comunicación cliente-servidor
gRPC	framework RPC tipado basado habitualmente en protobuf y HTTP/2
Concurrencia optimista	detección de cambios concurrentes sin bloquear por defecto

43. Referencias

[ASP.NET Core: fundamentos](#)

[API HTTP en ASP.NET Core](#)

[Referencia de Minimal APIs](#)

[API con controladores](#)

[Validación en ASP.NET Core](#)

[Errores en API y ProblemDetails](#)

[Inyección de dependencias](#)

[Configuración](#)

[OpenAPI en ASP.NET Core](#)

[Entity Framework Core](#)

[Rate limiting](#)

[CORS](#)

[SignalR](#)

[WebSockets](#)

[gRPC en .NET](#)

[Webhooks: conceptos y buenas prácticas](#)

[Semántica HTTP: RFC 9110](#)

[Problem Details: RFC 9457](#)

Revisión de versiones

Los ejemplos están orientados a .NET 10. Antes de iniciar una nueva edición del curso, revisa documentación, paquetes, cambios incompatibles y avisos de seguridad.